

GNN 推薦模型整合方案(討論用)

目的:把組員開發的 GNN 咖啡廳推薦模型([GNN/](#) 資料夾)整合進 CafeMatch 主系統。

本文件整理現況分析、可行的耦合方案與需要團隊決定的事項,供討論使用。

日期:2026-07-02

1. 現況盤點

1.1 GNN 端(組員的程式)

核心程式為 [GNN/new_step5_recommend.py](#) ,模型是 HGT(Heterogeneous Graph Transformer)異質圖神經網路,技術棧為 PyTorch + torch_geometric。

訓練資料:56 家花蓮咖啡廳 × 2,286 位 Google 評論者(爬蟲資料)。

推論有兩條路徑:

路徑	輸入	流程
舊用戶(圖中已有評論者)	該用戶的 BERT 向量	BERT 向量 → user_proj → HGT 圖卷積 → predictor 打分
新用戶(冷啟動)	心理測驗五維分數	五維 → quiz_projector(5→384)→ user_proj(384→128)→ 與咖啡廳 embedding 配對打分

打分後有分層抽樣機制:平均評分 ≥ 4.7 的「精選池」抽 3 家、4.0~4.7 的「優質池」抽 2 家,共推薦 5 家。

推論所需檔案: [best_model_with_quiz.pt](#) 、 [hetero_graph.pt](#) 、 [reviews_clean.csv](#) 、 [user2idx.json](#) 、 [cafe2idx.json](#) 、 [cafes_updated.json](#) 。

1.2 主系統端(CafeMatch)

- Flask + SQLAlchemy + MySQL,目前依賴很輕(無 torch / numpy / pandas)。
- 現有推薦入口:
 - Quiz 頁人格測驗**:算出五維分數(work/env/social/taste/cp),存入 [UserQuizResult](#) ,回傳人格類型文字,目前不直接推薦店家。
 - Chat 情境元件**:收集單次需求(時段/活動/同行/氛圍/餐飲/預算/特殊需求),轉成標籤權重 + 營業時間窗 + 價位範圍,由 [cafe_retriever._weighted_retrieve](#) 做硬過濾 + 標籤加權排序。
 - Chat 自由聊天**:關鍵字檢索 + LLM 回覆。

2. 關鍵發現

✓ 對得上的部分

1. 五維分數完全一致:GNN 新用戶路徑吃的 `work/env/social/taste/cp`,就是主系統 Quiz 頁算出並存在 `UserQuizResult` 的同一套五維。這是天然接口。
2. 咖啡廳 ID 一致:已比對 `GNN/cafes_updated.json` 與主資料庫備份(`backups/cafematch.sql`),id 對應同一批店家(1=Oops驚奇咖啡、2=原野牧場、3=小巷茉莉...)。GNN 輸出可直接 join 回主 DB 的 `cafes` 表。

⚠ 需要注意的部分

3. 舊用戶路徑對主系統目前不可用: `user2idx.json` 裡的 2,286 個用戶是爬蟲來的 Google 評論者,不是 app 的 `User` 表。真實註冊用戶不在圖裡,所以上線初期只有「新用戶(五維)路徑」能用。未來若把 app 用戶評論納入圖重新訓練,舊用戶路徑才有意義。
4. Chat 情境元件與 GNN 語義不同、不能直接互轉:
 - 情境元件問的是「這一次想要什麼」(今天下午、跟朋友、預算 200)→ 輸出標籤權重。
 - GNN 五維問的是「這個人是什麼樣的人」(長期偏好人格)。
 - 硬把情境答案映射成五維,分佈和語義都會失真(quiz_projector 是拿人格測驗分數分佈訓練的)。
5. 分層抽樣依賴評論統計:GNN 的推薦抽樣用到每家店的平均評分/評論數(來自 `reviews_clean.csv`),主 DB 沒有這些欄位,整合時需一併帶出或改用其他排序策略。

💡 混合使用的正確姿勢

五維分數不從情境元件來,從 Quiz 頁人格測驗來:

使用者填 Chat 情境選單

- └ 硬過濾:營業時間、價位(照舊)
- └ 標籤加權分:代表「這次的需求」(照舊)
- └ GNN 分數:若使用者做過人格測驗,取最近一次 `UserQuizResult` 五維
 - 代表「這個人長期會喜歡的店」(新增訊號)
- 混合排序(權重可調)

兩個訊號量測的東西不同,相加是互補。沒登入或沒做過測驗的使用者行為完全不變——GNN 是增強,不是依賴。

副作用(也可能是機會):chat 推薦的個人化程度取決於使用者是否做過人格測驗,可設計成引導動線(「做過測驗推薦更準」)。

3. 耦合方案比較

推論階段的一個重要事實:新用戶路徑在使用時不需要跑圖計算。咖啡廳的 128 維 embedding 對固定的圖是常數,新用戶分數只是三個小型 MLP(矩陣乘法),CPU 上毫秒級。

A. 離線匯出 + numpy

B. torch 進主 app

C. GNN 獨立微服務

做法	一次性腳本把咖啡廳 embedding + 三個 MLP 權重匯出成 .npz, 主系統用純 numpy 重現推論	主系統直接安裝 torch + torch_geometric, 載入 .pt	GNN 包成 FastAPI 服務(如 <code>/score</code> endpoint), 主系統 HTTP 呼叫
主系統新增依賴	只有 numpy	torch + torch_geometric(部署包 +1~2GB)	無(只多一個 HTTP client)
模型重訓時	重跑匯出腳本、換 .npz 檔	換 .pt 檔	GNN 服務自己換, 主系統無感
模型改架構時	✗ numpy 推論程式要跟著手改	改造過的 service 程式要同步	✓ 主系統完全不用動
部署	單一服務(現況)	單一服務但很肥	兩個服務(docker-compose), GNN 容器約 1~2GB
本地開發	一個 process	一個 process(Windows 裝 torch_geometric 常有麻煩)	兩個 process
舊用戶路徑/未來擴充	不支援(要重新匯出設計)	可支援	✓ 服務端保留完整能力, 重訓後直接可用
故障模式	無網路呼叫	無網路呼叫	要處理服務掛掉(fallback 回純標籤檢索)

判斷基準:模型還會不會動?

- **模型交付後就凍結** → 選 A, 最省事, C 的價值永遠不會兌現, 只留維運成本。
- **模型會持續迭代**(改架構、重訓、未來納入 app 用戶評論) → C 是對的投資, 組員可以完全獨立迭代, 不用碰主系統。
- B 基本上不建議: 承受了最重的依賴, 卻沒拿到 C 的隔離好處。

建議: 介面先行, A / C 可隨時互換

無論選哪個, 主系統都先定義一個窄介面:

```
# 主系統只依賴這個介面, 不關心背後實作
GnnRecommender.score(five_dim_scores: dict) -> dict[cafe_id, gnn_score] | None
# None = GNN 不可用, 呼叫端 fallback 回純標籤檢索
```

Chat pipeline 和檢索器只認這個介面。後面接 numpy 本地實作(A)還是 HTTP client(C), 隨時可換, 主系統其他程式一行不用改。就算先上 A、之後轉 C, 遷移成本也極低。

3.5 定位問題: GNN 是核心, 還是加分項?(需優先討論)

這是目前最關鍵的懸而未決事項, 建議團隊討論時**優先**於第 4 節的其他細節。

問題本質

GNN 設計之初的定位是「認識這個使用者,推薦適合他的店」——核心思路是系統要有一份代表這個人的資料。但目前唯一能餵給 GNN 的「認識使用者」資料來源,只有心理測驗(Quiz 頁),而心理測驗是**選填、非強制**的一次性動作。

結果是:GNN 從「推薦系統核心」變成「加分項」——只有主動做過心理測驗的使用者才吃得到 GNN 的效果,覆蓋率完全取決於心理測驗完成率(目前心理測驗本身也還沒接推薦,誘因偏低)。

這跟「GNN 是推薦系統一部分」的原始定位有落差,是設計走向分岔造成的:Chat 情境元件記錄的是「這一次」(用完即丟,沒有累積成使用者長期輪廓的設計),而 GNN 需要的是「這個人是誰」(需要某種長期識別使用者的機制)。目前系統裡唯一做到「長期識別」的只有心理測驗這一個選填動作。

三條路,各自的代價

	A. 提高心理測驗覆蓋率	B. 用行為資料推斷使用者輪廓	C. 承認現階段是加分項
做法	把心理測驗塞進新手引導、或 Chat 首次使用時邀請填寫	使用者收藏的店、Chat 情境元件選過的答案、瀏覽紀錄,累積換算成類似五維的分數,不需明確測驗	不假裝這次整合做到「核心」,先上線看資料,日後再評估
是否解決根本問題	✗ 治標不治本,使用者仍要額外做一件事才被 GNN 認識,對已在使用 Chat 的老使用者無回溯效果	✓ 每個有在用 app 的人自然累積資料,不需額外動作,才是真正符合「GNN 是核心」的做法	— 誠實縮小範圍,不解決,但不假裝解決
工程量	小(UI/引導流程)	大,現在完全沒有這套機制,需重新設計「行為→分數」規則	最小
現在能不能做	可以	時機未到 :GNN 目前訓練資料只有 2,286 個爬蟲評論者,app 真實使用者行為資料幾乎是零,就算做了推斷機制,短期也沒資料可餵,冷啟動問題只是換位置出現	可以
建議	可當低成本過渡動作	長期方向,建議累積一段時間真實使用資料後再啟動	現實的起點

建議討論順序

1. 先確認團隊(含指導老師)能接受「這次整合先做加分項,不是核心」這個範圍縮小,還是必須某種程度上兌現「核心」的定位。
2. 若能接受 → 採 C,可選擇性搭配 A(低成本引導)。
3. 若不能接受、堅持要做到「核心」→ 代表要追加 B 方案的工程範圍與時程,且要先評估「等多久才會有足夠的真實使用行為資料」,這可能影響整體專題時程規劃,需要及早跟老師溝通。

4. 需要團隊決定的事項

1. **模型的迭代計畫**:GNN 之後還會繼續改嗎?會不會把 app 真實用戶的評論納入重新訓練?
→ 這直接決定 A 還是 C。

2. (若選 C)誰維護 GNN 服務端？

→ 需要在 GNN 專案包一層 FastAPI(工作量不大,一個 endpoint),模型檔、依賴、容器歸維護者管。

3. (若選 C)部署方式:現況是單一 Flask 容器,選 C 變成 docker-compose 兩個服務,GNN 容器因 torch 約 1~2GB,部署環境放得下嗎？

4. 整合落點確認:第一階段先接 Chat 情境元件混合排序(已初步討論)。Quiz 交卷後直接推薦、Explore 頁個人化排序要不要排進後續？

5. 混合排序的權重:標籤分 vs GNN 分的比例(可先 50/50,上線後調)。

6. 分層抽樣要不要保留:GNN 原始程式有「精選池抽 3 + 優質池抽 2」的隨機性設計(每次結果略不同)。接進 chat 後要保留隨機抽樣,還是改成穩定的分數排序？

附錄:GNN 檔案清單速覽

檔案	用途	推論必需？
<code>best_model_with_quiz.pt</code>	最終模型權重(含 quiz_projector)	✓
<code>hetero_graph.pt</code>	異質圖資料(含 BERT 節點特徵)	✓
<code>reviews_clean.csv</code>	清理後評論(供平均分/已訪店排除)	✓
<code>user2idx.json</code> / <code>cafe2idx.json</code>	節點索引對照	✓
<code>cafes_updated.json</code>	店家詳細資訊	✓(可改為查主 DB)
<code>step1~4_*.py</code> 、 <code>train_with_quiz.py</code>	前處理與訓練腳本	✗(重訓才需要)
<code>best_GAT.pt</code> 、 <code>best_GraphSAGE.pt</code> 、 <code>best_HGT.pt</code>	模型比較實驗產物	✗
<code>.venv/</code>	組員的虛擬環境	✗(不應進版控,建議改成 requirements.txt)